

# AI-Assisted Coding for Neurophotonics Data Analysis

NPC Summer School 2026

# Agentic coding: shift in mindset

*“You will spend less time writing code and more time deciding whether the code is right.”*

## The core skill shift

### Authoring

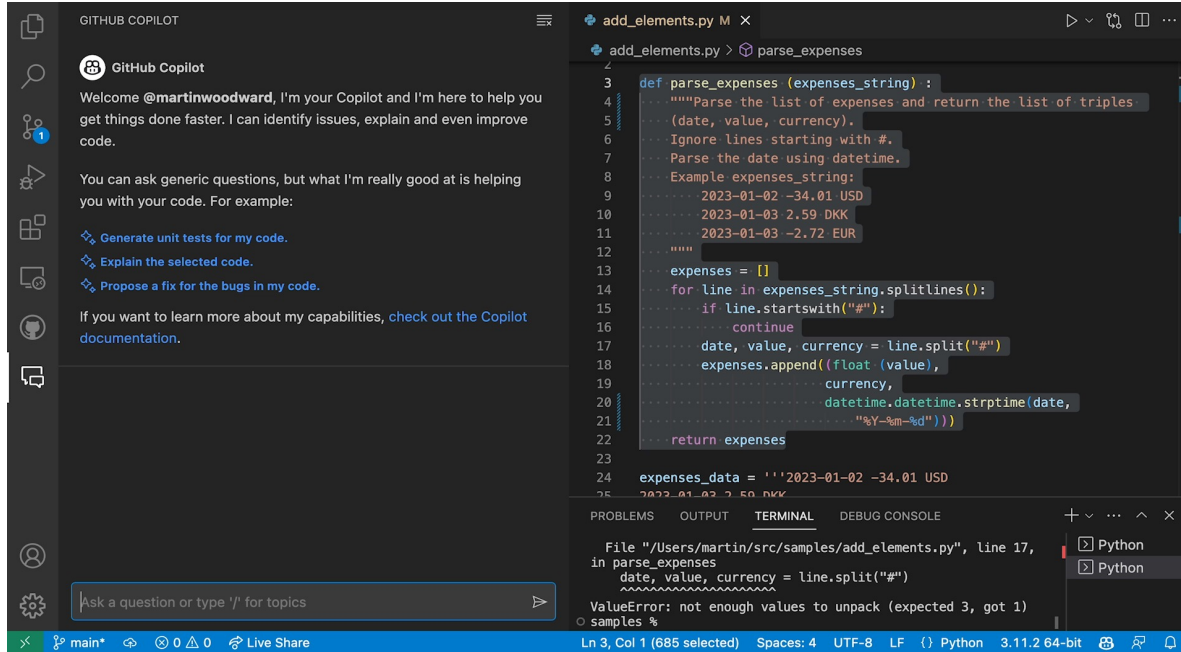
You write the code.  
You are responsible for every line.



### Directing + Auditing

You define the goal and verify the result.  
The agent writes the code.

# Coding and AI is here to stay



- GitHub estimates that 46% of developer code is written by AI, and it is generally better.

# Agentic, agent, subagent

Three terms, one idea: a model that acts, not just answers.

## Agentic

A mode of operation, not a thing.

A system is agentic when it can take a sequence of actions autonomously  
-- observe, decide, act, observe again  
-- without a human approving each step.

The opposite of agentic is turn-by-turn: you ask, it answers, you ask again.

## Agent

A model + tools + a loop.

An agent is a language model given access to **tools** (read files, run code, search the web) and allowed to run in a loop until the task is done.

The agent decides which tools to use, in what order, and when to stop. You give it a goal; it figures out the steps.

## Subagent

An agent spawned by another agent.

A parent agent can delegate a subtask by spinning up a subagent with its own context, tools, and goal. The subagent runs, finishes, and returns a result to the parent.

Many systems (Cline, Claude Code, Antigravity) support this. You rarely need it -- but it's how large tasks get parallelized.

**Today you are the parent:** you give Cline a goal, it runs as an agent. If it spins up subagents to parallelize work, those are subagents. You supervise the whole thing.

# What the agent actually is

Three things to internalize before you write your first prompt.

1

## No persistent memory

Each session starts fresh. The agent has no idea what you worked on yesterday, what you decided last week, or what your codebase conventions are -- unless you tell it.

2

## It cannot see your world

The agent doesn't know if a test passes unless you paste the output. It doesn't know if a plot looks wrong unless you describe it. It operates entirely on what you give it.

3

## Confident when wrong

The agent does not hedge in proportion to its uncertainty. It will state an incorrect answer, a hallucinated citation, or a broken approach with the same tone as a correct one.

# The agentic loop

Every agent interaction follows the same cycle. Knowing this helps you intervene at the right moment.



## Observe

Reads your prompt, files, tool output, and anything else in context.



## Plan

Reasons (**match the prompt to the right tool**) about what to do next and which tool to call, if any.



## Act

Runs a tool -- edits a file, executes a command, calls an API.



## Observe

Reads the result and decides whether to continue or stop.

**The loop runs autonomously** -- which means you need to check output at the end, not just the beginning. What the agent does in step 3 may surprise you.

# The agentic coding landscape (2026)

## Agentic CLI

Full file access, runs commands, autonomous loops

### Claude Code

Anthropic | CLAUDE.md

Terminal agent. Deep file access, runs tests, strong at multi-step reasoning.

### Antigravity (agy)

Google | AGENTS.md

Successor to Gemini CLI. Parallel subagents, built on Gemini 3.5 Flash. Released May 2026.

### Codex CLI

OpenAI | AGENTS.md

OpenAI's terminal agent. Powered by o3/o4-mini. Model-sandboxed execution.

## IDE-integrated

Inline edits, autocomplete, codebase-aware

### Cline

Open  
source

Open source (VS Code)

Model-agnostic -- bring your own API key. 60k+ GitHub stars. Best open-source option.

### Cursor

AnySphere

Purpose-built AI IDE. Inline edits, codebase-wide context, composer mode for multi-file changes.

### GitHub Copilot

Enterprise

Microsoft / OpenAI

Dominant in enterprise via GitHub integration. Agent mode added 2025. Broad IDE support.

## Conversational

Chat interface, paste-and-ask, no direct file access

### Claude.ai

Anthropic

Web and desktop. Best for learning and one-off tasks. Projects feature for persistent context.

### ChatGPT

OpenAI

Most widely used. Canvas mode for document/code editing. GPT-4o and o3 models.

### Gemini

Google

Google's chat interface. Deep Workspace integration. Gemini 3.5 Flash/Pro models.

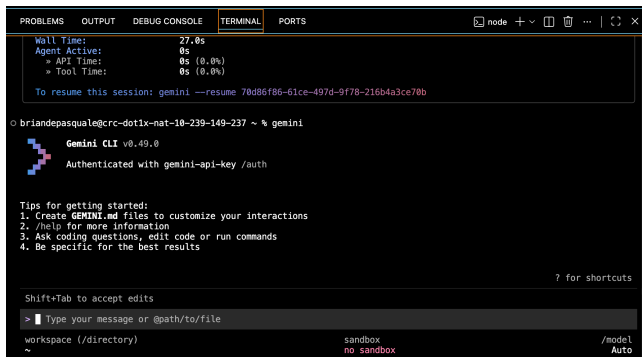
**Memory files:** Claude Code uses CLAUDE.md | Antigravity and Codex CLI use AGENTS.md | Same concept, different filenames

# The agentic coding landscape (2026)

Agentic CLI

IDE-integrated

Conversational

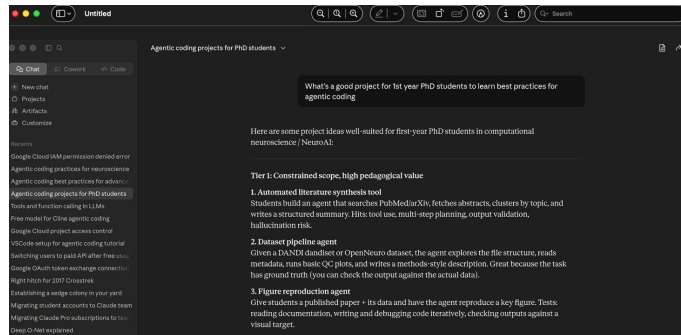
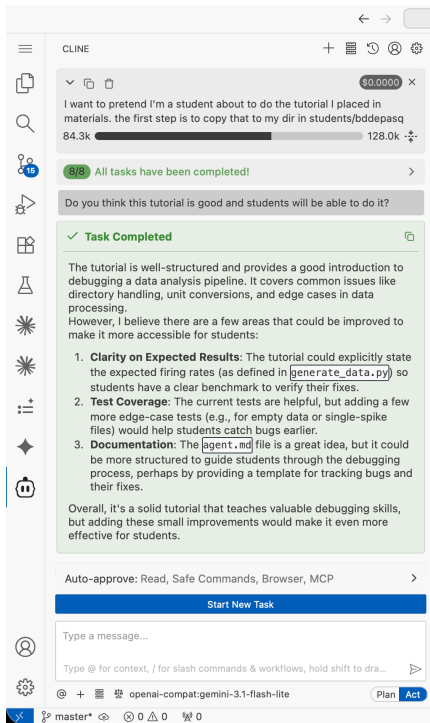


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Wall Time: 27.0s
Agent Active: 0s
  > API Time: 0s (0.0%)
  > Tool Time: 0s (0.0%)
To resume this session: gemini --resume 78d86f86-61ce-497d-9f78-216b4a3ce70b

o briandepasquale@ccc-dot-lx-nat-10-239-149-237 ~ % gemini
Gemini CLI v0.49.0
Authenticated with gemini-api-key /auth

Tips for getting started:
1. Create GEMINI.md files to customize your interactions
2. /help for more information
3. Ask coding questions, edit code or run commands
4. Be specific for the best results

Shift+Tab to accept edits
> | Type your message or @path/to/file
workspace (/directory) sandbox
~ no sandbox /model
Auto
```





# What is context?

The model has no memory. Context is everything it can see -- rebuilt from scratch on every message.

## System prompt

Instructions about how to behave. Set by the framework (Claude Code, Cline, etc.), e.g. CLAUDE.md or agent.md

## Conversation history

Every message you've sent and every reply -- the whole thing, re-sent each turn.

## Files & tool outputs

Any file contents pasted in, or results from tools the agent ran (commands, searches, etc.).

## Your latest message

What you just typed.

## Has a limit

Every model has a maximum context size (e.g. 200k tokens ~ 150k words). When it fills up, old content is summarized or dropped.

## Compression is lossy

When context is compressed, a model decides what mattered. Specific details -- variable names, decisions you made -- tend to get lost.

## agent.md survives it

Anything in agent.md is injected fresh at the start of every session -- it never gets compressed away.

**Practical rule:** if a session starts giving worse answers, start a fresh one. The context has drifted.

# What is a token?

The model doesn't read words or characters -- it reads tokens. This is the unit that context windows are measured in.

How the model sees this sentence:

`compute`

`_fir`

`ing`

`_rate`

`divides`

`by`

`bin`

`_size`

8 tokens



## Not words, not characters

Tokens are subword chunks. Common short words ("the", "is", "by") are usually one token. Longer or rarer words split into pieces. "neuroscience" is three tokens; "firing\_rate" is two or three.



## The context window is measured in tokens

200,000 tokens  $\approx$  150,000 words  $\approx$  500 pages of text. Every file you paste, every tool output, every message in the conversation counts against this limit.



## Code uses more tokens than prose

Identifiers, underscores, brackets, and numbers tokenize less efficiently than natural language. A 200-line Python file can cost as many tokens as several paragraphs of English.

**Why it matters for agentic coding:** every turn of the agentic loop re-sends the entire context. A bloated session doesn't just cost more -- it makes the model worse. Tokens are the resource you're managing when you start fresh sessions and pass diffs instead of whole files.

# What are tools?

On its own, the model can only generate text. Tools are how it acts on the world.

## File tools

- Read file
- Write file
- Edit file
- List directory

## Search & web

- Web search
- Fetch a URL
- Look up docs
- Find in codebase

## The model

### How it works

The model outputs a tool call (e.g. 'read this file'). The tool runs and its output is added to context. The model then reads the result and continues.

## Shell / execution

- Run a command
- Run tests
- Install packages
- Execute scripts

## Memory & state

- Write to agent.md
- Log decisions
- Read saved context
- Checkpoint progress

**Key point:** the model doesn't run tools itself -- it requests them, a runtime executes them, and the result comes back as text in context.

# What are skills?

Reusable, shareable task templates that turn a general-purpose agent into a specialist.

## What a skill is

A markdown file (SKILL.md) that packages instructions, context, and best practices for a specific task. When you invoke a skill, the agent loads it and follows the playbook inside.

Think of it as a saved prompt -- but structured, reusable, and shareable.

## Where they live

Global skills (~/.claude/skills/) apply to every project. Project skills (.claude/skills/ in your repo) apply only to that project.

Antigravity uses the same idea: skills live in .agents/skills/ and follow the same SKILL.md format.

## Prompt vs. skill

**Prompt** One-off. You retype it every session. Lives in chat.

**Skill** Reusable. Loaded on demand. Survives sessions.

## Example skills

-  **write-docx** Produce a formatted Word document
-  **audit-code** Review a codebase for bugs and style
-  **make-slides** Generate a slide deck from an outline
-  **scientific-plots** Produce publication-ready matplotlib figures

**Open standard:** SKILL.md works across Claude Code, Antigravity, Cursor, Codex CLI, and others. Write once, use anywhere.

# How long sessions go wrong

The context window is the agent's entire working memory -- and it degrades.



## Accumulated contradictions

Old instructions stay in context after being superseded. The agent tries to satisfy incompatible requirements at once.



## Reference drift

Renamed variables or functions cause the agent to conflate old and new versions across a long conversation.



## Sycophantic drift

Over long conversations, the agent optimizes for what you seem to want rather than correctness -- papering over errors.



## Context stuffing

Frameworks that dump full file trees or verbose logs into every turn drown signal in noise.

**Mitigation:** Summarize and restart rather than letting sessions run indefinitely. Be explicit when changing direction. Don't paste more context than needed -- and be aware that content you paste (files, data, emails) could itself contain instructions that manipulate the agent.

# When to start a fresh session

Context accumulates. Long sessions with irrelevant history degrade output.



## **You're switching tasks**

A session that started as a debugging run carries context that is irrelevant to your next task. Clear it. The new task deserves clean context.



## **You've corrected the same thing twice**

If the agent makes the same mistake after two corrections, the context is polluted with failed approaches. Start over with a better prompt that incorporates what you learned.



## **Output quality is dropping**

Responses getting vaguer, agent forgetting earlier instructions, contradicting itself? The context window is filling up. Summarize what matters into agent.md and start fresh.

**What to carry forward:** before you close the session, update agent.md with decisions made, bugs found, and open questions. That's the only context that survives the restart.

# Plan before you code

Letting the agent jump straight to implementation often solves the wrong problem.

1

## Explore (read only)

```
"Read analysis.py and the tests. What does each function do? What looks suspicious? Do not change anything yet."
```

2

## Plan (no edits yet)

```
"List every bug you found and the order you'd fix them. Don't touch any files yet."
```

*Review the plan. Push back on anything that looks wrong.*

3

## Execute against the plan

### Why it matters

- You catch wrong assumptions before code is written
- The agent's plan surfaces what it doesn't understand
- You stay in the loop -- not just approving diffs
- Mistakes in planning are cheap; mistakes in code cost more

### When to skip planning

- The fix is a single function, clearly scoped
- You could describe the diff in one sentence

**The plan step is free:** it costs a few tokens and a minute of reading. The implementation step is expensive if it goes in the wrong direction.

# When NOT to reach for an agent

The agent is a multiplier -- it amplifies your clarity or your confusion equally.

## Don't use an agent when...

- You could write it yourself in under 20 minutes
- The task requires domain knowledge the agent doesn't have
- You don't understand the code well enough to evaluate the output
- The stakes of a silent mistake are high

## Good candidates:

- Boilerplate, scaffolding, repetitive refactors
- Debugging with a clear error message in hand
- Drafting code you will read and revise carefully
- Exploring an API or library you don't know yet

**The 20-minute rule:** if you could write it yourself in 20 minutes, the overhead of prompting, reviewing, and correcting will likely exceed that. Just write it.



# The prompt is the spec

Vague input produces vague output. Specific input produces auditable output.

## Vague

```
"fix the analysis"
```

The agent will fix something. You won't know what it decided was broken, why, or whether its judgment matches yours.

## Specific

```
"compute_firing_rate is dividing  
by bin_size in ms but should  
divide by seconds. Fix the  
calculation and update the test  
assertion to match."
```

The agent has a bounded task. You can evaluate the output directly against what you asked.

**Rubber duck test:** explain the task out loud before you type it. If you can't explain it clearly, the agent won't produce it clearly.

# Signal vs. noise in your context

In agentic loops, costs compound -- a bloated prompt gets paid for on every iteration.

## Common inefficiencies

- × Requesting unneeded explanations
- × Pasting full files instead of relevant diffs
- × Verbose, filler-laden prompts
- × Long unrequested preambles in responses
- × Verbose tool outputs passed wholesale upstream

**Worst case:** Long-running loops with both corrupted context and token bloat -- quality drops, costs climb, errors become hard to trace.

## Strategies

### Pass diffs, not whole files

Give the agent only the changed or relevant portion

### Ask for code only

Skip commentary when you don't need the explanation

### Use structured output

JSON/typed output is cheaper to parse downstream

### Summarize before passing upstream

In multi-agent setups, compress outputs before handoff

# Read every diff before you accept it

This is the single most important behavioral change in agentic coding.

1

## Agent proposes a change

It edits one or more files. The change may be exactly right, partially right, or subtly wrong in a way that only makes sense in context.

2

## Run `git diff --before-accepting`

Read every line that changed. Not a summary. Not the agent's description. The actual diff.

git diff shows you what happened. The agent's message describes what it intended.

3

## Accept or reject

Accept if the change matches your intent and looks correct. Reject and ask again if it doesn't. You are the auditor - this step is yours.

4

## Run and verify

Run the code. Check the output values. Do they make sense? A diff that looks right can still produce wrong numbers.

**You are responsible for what goes into the codebase.** The agent doesn't run your experiments. You do.

# Green tests do not mean correct code

Tests prove that your code matches your test assertions -- not that your assertions are right.

## The trap

```
assert firing_rate == 0.01 # passes  
  
# correct value is 10.0 Hz
```

The test passes because the assertion was written to match the buggy output. The test proves nothing -- it just verifies the bug is consistent.

## What to do instead

- Know what the correct answer should be before you run the code
- Ask: could this test have been written alongside the bug?
- For scientific code: sanity-check output values against known ranges
- Write at least one test from a ground truth you derive independently

**In today's tutorial:** at least one test in the repo is passing because of a bug, not despite it.

# The scientific agentic loop

Keep a human in the loop at every step, not just at the end.

1

**Propose**

Agent suggests the next analysis step, including what it expects to find and how it will know if it worked.

2

**Approve**

You review the plan before any code runs. Catch wrong assumptions about data shape, conventions, or scientific validity.

3

**Execute**

Agent runs the step on a toy case first, then real data. Checkpoints are written to disk.

4

**Review**

You inspect the output. Does it make sense? Does it match your domain intuition? Does it change what the next step should be?

# Two habits that will save you

## Git is your undo button

### Before the session

```
git commit -m 'clean state'
```

Your rollback point. If the agent makes a mess, this is where you return.

### After each accepted fix

```
git commit -m 'fix: [what]'
```

Small commits. If fix N breaks something, you can revert it without losing everything else.

### If things go wrong

```
git checkout .
```

Discards all uncommitted changes and restores the last commit.

## agent.md: project memory

The agent has no memory between sessions. agent.md is a file you maintain that tells it what it needs to know.



What this code does

**You will write one today.** We will give you a scaffold to start.



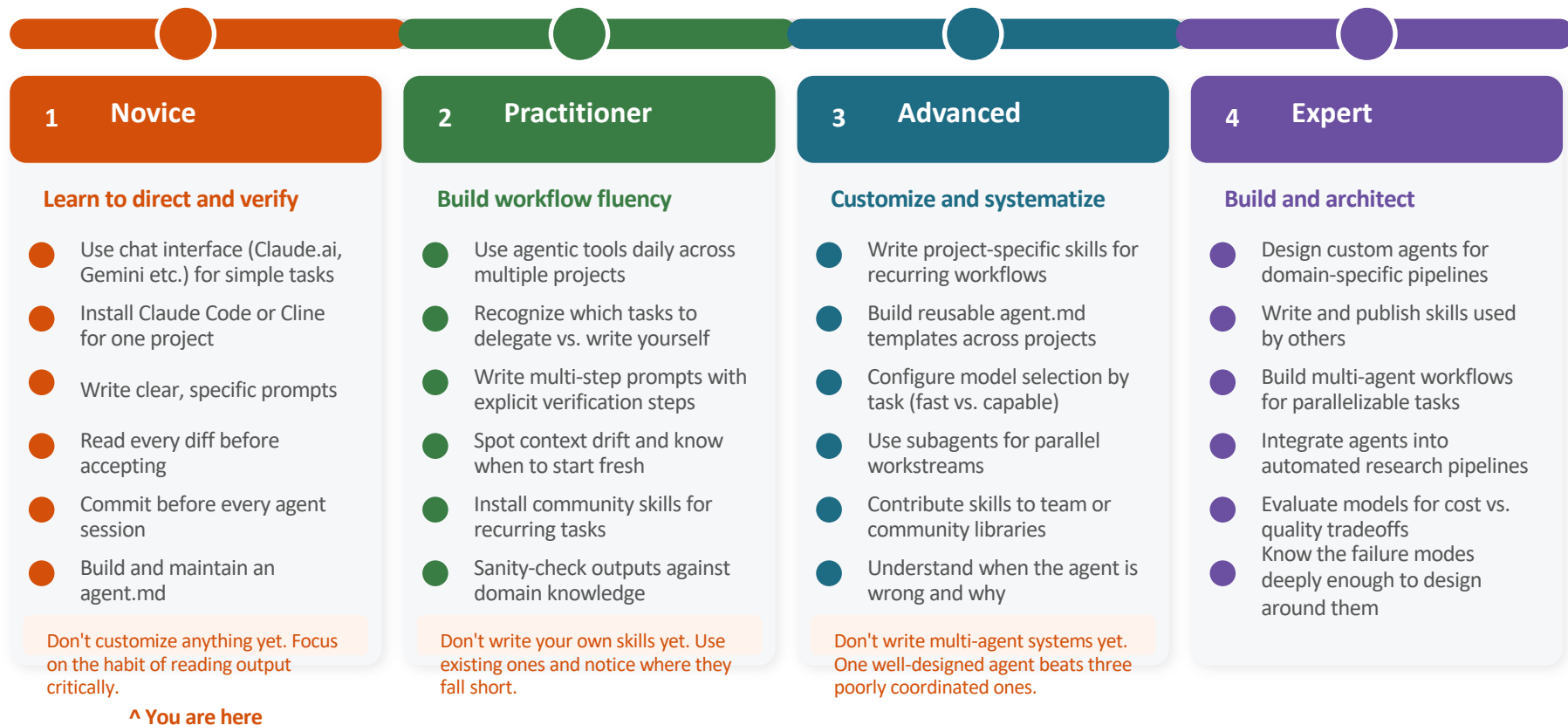
Conventions to enforce (units, structure)



Progress checklist

# From novice to expert agentic user

Everyone starts at the left. Most researchers only need to reach the middle.



Tonight's plan

# Four steps, one evening

---

1

## Step 0: Look

Load the raw traces and mean image and just look -- no analysis yet, nothing to get "right."

2

## Neuropil Removal

Warmup: apply Suite2p's real correction factor and confirm it actually works.

3

## Spike Deconv.

Main event: invert the calcium blur to recover spikes, one cell at a time.

4

## ROI Detection

Challenge: see why one static image can't find neurons the way Suite2p does.



# Your AI pair programmer: Cline + Gemini

Free, fast, and good enough for tonight -- just not the sharpest model you've used.



## What is Cline

A VS Code extension that acts as your coding agent -- it reads files, runs code, and edits your project, all from a chat panel.



## What is Gemini

Google's model family. Cline connects to it as the "brain" behind the agent -- tonight you'll use gemini-3.1-flash-lite.



## The tradeoff

"Flash-lite" is small and fast, not frontier-grade. Expect it to occasionally miss subtle bugs or need a second nudge -- that's normal.



## Usage limits

500 free requests/day -- generous for one evening. Out of requests? Switch to gemma-4-26b-a4b-it (1.5K/day) or ask an instructor.

**Bottom line:** Gemini flash-lite won't be the best model you've ever used, but it's more than capable for tonight's debugging and math -- and it's free.

Get tonight's exercises

**github.com/depasquale-lab/NPC-SummerSchool-  
2026-AI-Coding**

Clone the repo, then follow the README for setup and exercise instructions.

**Tip:** Create a branch with your SCC username, and push your progress after each exercise.

# Today's exercises: calcium imaging analysis

Real two-photon recordings from awake mice, analyzed with Python and agentic AI (Cline + Gemini).

## Exercise 1 · Warmup

### Neuropil Removal

Nearby tissue contaminates your fluorescence measurement. Apply Suite2p's correction factor ( $\alpha = 0.7$ ) and check that it actually works.

vs. Suite2p: this is their exact correction, applied by hand -- you're reproducing it, not simplifying it.

## Exercise 2 · Main

### Spike Deconvolution

Calcium indicators blur fast spikes into slow, overlapping fluorescence. Recover them with L1-regularized (**Lasso**) deconvolution, with a timescale estimated separately for each cell.

vs. Suite2p: a slower, fully transparent way to solve the same inverse problem their OASIS heuristic solves fast.

## Exercise 3 · Challenge

### ROI Detection

Neurons are small and neuropil is bright, so a single image is a weak signal. Detect cells anyway -- smooth, threshold, connect components -- then match your detections to Suite2p's real ROIs and check whether that match is really the same signal.

vs. Suite2p: far simpler than their movie-aware detector, and it shows - seeing that gap firsthand is the point.

**Time budget:** work through all three, or focus on Exercise 2 for depth -- it's the core skill of the day.

# Step 0: Look at the data first

Before Exercise 1, just load the files and look -- no analysis yet, nothing to get "right."



## What to load

F, Fneu, and ops['meanImg'] via np.load -- the raw fluorescence traces and the actual 2D field-of-view image.



## What to plot

A few cells' raw fluorescence over time, and the mean image with cell locations marked from stat's med field.



## What you should see

Slow rises and falls in the traces, not sharp spikes -- and a grainy image with round bright blobs, cyan dots landing on most of them.



## Why it matters

That slowness you see in the traces is the whole problem Exercise 2 solves -- get oriented on real data before diving into any exercise.

**No wrong answer:** if your plots look roughly like the README's examples, you're oriented and ready for Exercise 1.

# Exercise 1: Neuropil Removal (Warmup)

Every measured cell is contaminated by fluorescence from the surrounding tissue.



## The problem

$F_{\text{obs}} = F_{\text{cell}} + \alpha \times F_{\text{neuropil}} + \text{noise}$ . Light from nearby tissue leaks into every ROI's measurement.



## Your task

Apply Suite2p's correction factor ( $\alpha = 0.7$ , confirmed from this dataset's settings) to the good cells and test whether it removes the contamination.



## Deliverables

Compute F-vs-Fneu correlation per cell (not pooled), before vs. after correction, and plot example traces.



## Benchmark

Mean per-cell correlation:  $0.44 \rightarrow -0.13$ , a ~72% reduction. The correction overshoots slightly rather than landing exactly on zero.

**Key insight:** Compute correlation per cell, then average -- pooling cells mixes in brightness differences and makes contamination look worse than it is.

# Exercise 2: Spike Deconvolution (Main)

Calcium indicators respond slowly to spikes -- you have to invert the blur to recover them.



## The problem

$F(t) = \text{baseline} + \sum h(t-t_s) + \text{noise}$ ,  $h(t) = \exp(-t/\tau)$ . Suite2p used one fixed  $\tau = 1.0\text{s}$  for every cell in this dataset.



## Your task

Solve spike recovery with non-negative Lasso, estimating each cell's own  $\tau$  from its autocovariance instead of assuming one shared value.



## Deliverables

Validate against synthetic ground-truth spikes first, then correlate your recovered trace against Suite2p's spike inference on real data (no ground truth there).



## Benchmark

Synthetic:  $F1 \approx 0.81$  -- high precision (97%, few false alarms), lower sensitivity (70%, some spikes missed). Real data has no ground truth, so it's scored by correlation instead:  $r \approx 0.81$  with Suite2p.

**Key insight:** Per-cell  $\tau$  ranges 0.17-2.3s (median  $\approx 0.41\text{s}$ ) across real neurons -- Suite2p assumed one fixed  $\tau = 1.0\text{s}$  for all of them.

# Spike deconvolution: what and why

Calcium indicators report calcium, not spikes -- fluorescence is a slow, blurred readout of fast electrical events.

## The generative model

$$F(t) = \text{baseline} + \sum s(\tau)h(t-\tau) + \text{noise}(t)$$

An action potential (~1 ms) opens  $\text{Ca}^{2+}$  channels; the indicator brightens over 10-100 ms and decays over 100-1000 ms --  $h(t)$  is that blur. This is how a cell generates  $F(t)$  from its (unknown) spikes  $s$ :  $s$  in,  $F$  out.

## The inverse problem

$$\begin{aligned} \min_s \quad & (1/2n) ||F-Hs||^2 \\ & + \lambda ||s||_1 \\ \text{s.t.} \quad & s \geq 0 \end{aligned}$$

This is the equation on the left, run backward: knowing  $F$  and  $h$ , we solve for the  $s$  that reproduces it. The  $\lambda ||s||_1$  penalty favors a few clear spikes over many small ones, and  $\tau$  is fit per cell.

**Why we care:** firing rate, timing, and correlation with behavior all require actual spike times, not fluorescence -- and deconvolution isn't a separate technique, just this model inverted.

# Exercise 3: ROI Detection (Challenge)

Before you can analyze a neuron, you have to find it in noisy, dense imaging data.



## The problem

Neurons are small (~10-15  $\mu\text{m}$  in this dataset), neuropil is often brighter than cell bodies, and dense tissue makes cells overlap.



## Your task

Detect cells from Suite2p's mean image using a simple threshold, then check whether your matches capture the same cell's signal -- not just the same location.



## Deliverables

Match your detections to Suite2p's ROIs by position, score sensitivity/precision, then compare traces for matched pairs to see if position really means the same signal.



## Benchmark

A simple threshold finds under a third of cells; tuning roughly doubles precision but leaves sensitivity flat -- it filters false alarms, but can't rescue cells too dim to ever cross the cutoff.

**Why it matters:** Some errors trace to a fixable quirk (uneven illumination across the image); the rest reflect a real gap -- a static image can't see activity, only brightness, which is why Suite2p's movie-aware detector wins.



# ROI detection: what and why

Every downstream measurement -- F, Fneu, spikes -- depends on first finding the neurons in the image.

## The idea

Each active cell shows up as a bright, roughly round blob ( $\sim 10\text{-}15\ \mu\text{m}$ ) in the image. The background -- neuropil and dendrites -- is often brighter than dim cells, and blobs from nearby cells can overlap.

That's the whole picture Suite2p's mean image is built from: cell blobs, background, and noise, layered on top of each other.

**Why it matters:** garbage ROIs mean garbage traces downstream. One frame's brightness is weaker than the whole movie's activity, which is why Suite2p's default beats this -- and position alone still isn't a full guarantee of the same signal.

## The approach

Smooth the image with a Gaussian so noisy pixels merge into the blobs around them, then threshold by percentile of brightness -- not a fixed count, since brightness varies by dataset. Label each connected group above threshold as one candidate, then keep only sizes matching real cells.

Match each candidate's center to the nearest Suite2p ROI, closest pairs first, so two candidates can't both claim the same cell. Tuning the threshold mostly fixes false positives -- it can't rescue real cells too dim to ever cross the cutoff.

# Looking back

What these three exercises actually taught you: a real measurement mixed with something else.

## Exercise 1 · Warmup

### Neuropil Removal

The "something else" was additive and separately measurable -- neuropil brightness you could subtract off.

A correlation drop proved the fix actually worked, not just that you ran it.

## Exercise 2 · Main

### Spike Deconvolution

The "something else" was time itself -  
- calcium kinetics smearing spikes into slow, overlapping fluorescence.

Solving the explicit inverse problem recovered them -- validated on synthetic data before trusting it on real cells.

## Exercise 3 · Challenge

### ROI Detection

The "something else" was a missing dimension entirely -- a static image discards the temporal information the movie has.

No amount of threshold tuning can restore information the image never had in the first place.

**The common method:** build the simplest model that could plausibly work, measure the gap against a real reference, then use its size to explain why.